

Allgemeine Projektbeschreibung

Ziel dieses Projektes ist die praktische Umsetzung und Vertiefung der in der Theorie vermittelten Kenntnisse auf den Gebieten

- Lexikalische Analyse (Scanner)
- Syntaktische Analyse (Parser)
- Attributierte Grammatiken
- Semantische Analyse
- Symbollistenverwaltung
- Zwischencodeerzeugung
- Codeerzeugung
- Umgang mit dem Compiler-Generator Coco/R.

Programmiersprache MiniIEC (MIEC)

Gegeben sei eine einfache Sprache namens MIEC in folgender Darstellung:

MIEC → **PROGRAM** *ident*
 (*VarDecl*)?
 BEGIN
 Statements
 END

VarDecl → **BEGIN_VAR**
 ident : **Integer** ; (*ident* : **Integer** ;)*
 END_VAR

Statements → *Stat* (*Stat*)*
Stat → *ident* := *Expr* ;
 | **print** (*Expr*) ;
 | **WHILE** *Condition* **DO** *Statements* **END**
 | **IF** *Condition* **THEN** *Statements* **END**
 | **IF** *Condition* **THEN** *Statements* **ELSE** *Statements* **END**

$Expr \rightarrow Term (+ Term)^*$
 $Expr \rightarrow Term (- Term)^*$

$Term \rightarrow Fact (* Fact)^*$
 $Term \rightarrow Fact (/ Fact)^*$

$Fact \rightarrow ident$
 $Fact \rightarrow number$
 $Fact \rightarrow (Expr)$

$Condition \rightarrow Expr Relop Expr$
 $Relop \rightarrow =$
 $Relop \rightarrow <=$
 $Relop \rightarrow >=$
 $Relop \rightarrow !=$
 $Relop \rightarrow <$
 $Relop \rightarrow >$

Als Datentyp ist in der ersten Ausbaustufe nur der Typ Ganzzahl (Integer mit 4 Byte) erlaubt. Kommentare werden zwischen (* und *) geschrieben. Die Standardprozedur `print` gibt den Ausdruck `expr` auf der Konsole aus und bewirkt einen Zeilenvorschub. Die Namen im Fettdruck sind sogenannte Schlüsselwörter.

Beispielprogramm in MIEC

Jedes MIEC-Programm befindet sich in einer separaten Datei mit der Erweiterung `*.miec`.

```
1 PROGRAM Hello
2   BEGIN_VAR
3     a: Integer;
4     b: Integer;
5   END_VAR
6 BEGIN
7   a := 3 * 6 + (2 * 3);
8   WHILE b < a DO
9     b := b + 1;
10  END
11  IF a > b THEN
12    print(a);
13  ELSE
14    print(b);
15  END
16 END
```

Teil 1: Scanner und Parser

Das Ziel der ersten Übung besteht darin, einen Scanner (Lexer) und einen Parser soweit vorzubereiten und zu generieren, dass dieser MIEC-Programme übersetzen kann und syntaktische Fehler erkennt. Als Compiler-Generator wird Coco/R verwendet, machen Sie sich dazu mit der Dokumentation auf der Kommunikationsplattform vertraut (*Übersicht_CocoR.pdf* und *CocoR_Tutorial*). Zusätzlich steht ein Beispiel `Taste.zip` zur Verfügung. Führen Sie anschließend folgende Implementierungsschritte durch:

1. Installation des *MS Visual Studio*-MIECCompiler-Projektes.
2. Schreiben Sie eine attributierte Grammatik `MIEC.atg` entsprechend der Programmiersprache MIEC, die mit dem Compiler-Generator Coco/R verarbeitet werden kann. Coco erzeugt in der C++-Version einen Parser (`Parser.h`, `Parser.cpp`) und einen Scanner (`Scanner.h`, `Scanner.cpp`).

Aufruf von Coco/R:

```
Coco.exe <ATGFilename.atg> -o <DirOfGeneratedFiles> -namespace <CompilerNamespace> -frames <DirOfFrameFiles>
```

3. Aufruf des Compilers:

```
MIECCompiler.exe -in <file.miec> -out <file.iex>
```

Die Aufrufchnittstelle ist unbedingt einzuhalten, um ein Zusammenspiel mit der automatischen Testplattform (ACOTEP) zu gewährleisten. Übergeben Sie dem MIECCompiler den Pfad der Quelldatei (`FileName.miec`) und der Zieldatei (`FileName.iex`) über die Kommandozeile. Prüfen Sie die Dateien auf die entsprechende Dateierweiterung `.miec` bzw. `.iex` und rufen Sie im Anschluss den Scanner und Parser auf.

4. **Ergebnisdatei:** Der Compiler erzeugt eine Datei `MIECCompiler.report` im ASCII-Format, die Informationen über den Compiliervorgang speichert. Existiert die Datei nicht, wird sie erzeugt, existiert sie, wird das Ergebnis an die bestehende Datei angehängt! Für einen Compiliervorgang wird ein Eintrag in der Reportdatei eingetragen. Folgendes Beispiel zeigt das genaue Dateiformat:

```
Thu Sep 12 12:31:17 2022 => ..\SourceFiles\xfail_Fehler_noVar.miec: FAILED: 1 error(s) detected
Thu Sep 12 13:01:04 2022 => ..\SourceFiles\Mul.miec: OK
Thu Sep 12 13:06:35 2022 => D:\fh-hagenberg\Projekt_Codeerzeugung\src\WhileTest.miec: OK
```

5. **Ausgaben:** Die Ausgaben in die Ergebnisdatei werden im gleichen Format auf **stdout** (kein Compilerfehler) und auf **stderr** (es treten Compilerfehler auf) ausgegeben.
6. Schreiben Sie verschiedene MIEC-Testprogramme und testen Sie den MIECCompiler ausführlich! Testdateien, die bei der Übersetzung einen Fehler liefern sollen, müssen mit dem Präfix "xfail" beginnen (Grund: ACOTEP).
7. Kommentieren Sie Ihren Quellcode und verwenden Sie Doxygen-Kommentare!